

INF-2700: Database systems

Assignment 1

Jens Christian Valen Leynse

J1e040

j1e040@post.uit.no

Sep 25th, 2019

Introduction

The assignment is to perform SQL queries in a given database through a SQLite shell and implement a tiny SQL shell in C. The assignment is split into three parts where the first part is for the user to setup the organizations of the files. The next part consists of three subparts as they are all centered around SQL. The first is to describe the schematic given in the assignment, the second is to simply run the given SQL queries in the terminal, and the third is to implement a new SQL query given the specifications. The last task of the assignment is to implement a database application in C capable of running SQL queries interactively and presenting the result in a JSON format.

Requirements:

- Create your own database “shell”.
- Present results as JSON

Technical Background

SQLite is a C-language library which is the most used database engine in the world. It differs to other database management systems in that it is embedded into end program. It does have some integrity flaws as conversion between formats is not guaranteed, but since it's a standalone database management system it benefits from avoiding the passing of data to another process increasing its efficiency.

PART I

Task 1. Database schema

Describe the inf2700_orders database schema.

The inf2700_orders database schema is a database schema set up as 8 different tables, all containing attributes of their own which can either be empty or not. The attribute also

specifies whether the text, integer, etc. ends with a NULL to mark its end. They include keys that references other tables.

The customers table consists of information regarding the customers private information. This includes information like names, location, phone number and credit limit.

The employees table includes their number, name, title, mail address and information regarding office details. And a foreign key referencing the employees table.

The offices table includes location, phone number and the office identification.

The order details table include every detail for the buyer so; order number, product identification, amount ordered as well as the price and the line plus a primary key referencing order number and product code, as well as a foreign key referencing the products table.

The orders table includes; when the order was made, required, shipped, what the status of the order is and who the buyer was as well as some comments on the shipment and a foreign key to the customers table.

The payments table includes the customer, the payment amount as well as the date the purchase was made with the check number. The tables primary key is set to the customer number and the check number, and the foreign key references the customers table.

The products table includes mostly product details like id, name, product line, scale, vendor a description and also the quantity in stock with the price plus a foreign key to the product lines table.

Lastly the product lines table only has two attributes namely the product line and a description for the line.

Task 2. Run the given SQL queries

Run the queries listed below, and for each query, write in your own words a few lines that describe its purpose and how it works.

a)

```
SELECT  C.customerName, C.contactLastName, C.contactFirstName
FROM    Customers C;
```

Description: Fetches the costumers name, and the contacts first and last name from the customers query

Implementation: Retrieves the costumer name, contacts last name and contacts first name attributes from the costumers database query under the abbreviation C

b)

```
SELECT  *
FROM    Orders O
WHERE   O.shippedDate IS NULL;
```

Description: Fetches all the objects in the orders query where the shipped date is empty

Implementation: Fetches all the data from the Orders database query and abbreviates the name as O. Filters the results to queries where the shipped date attribute from the orders query is empty

c)

```
SELECT  C.customerName AS Customer, SUM(OD.quantityOrdered) AS Total
FROM    Orders O, Customers C, OrderDetails OD
WHERE   O.customerNumber = C.customerNumber
        AND      O.orderNumber = OD.orderNumber
GROUP BY      O.customerNumber
ORDER BY      Total DESC;
```

Description: Fetches the costumers who made a specific order and sorts them by how much they ordered

Implementation: Fetches customer names from costumers query to the alias customer, fetches the sum of the items in the quantity ordered attribute in the order details query to the alias Total. Sets the orders, costumers and order details database queries to abbreviated characters. Filters the results to only include results where the costumer number attribute is the same from the orders query and the costumers query and results where the order number attribute is the same in the orders query and the order details query. Arranges the identical customer numbers from the orders query into groups. Sorts the results in a descending numerical order using the alias total

d)

```
SELECT  P.productName, T.totalQuantityOrdered
FROM    Products P NATURAL JOIN
        (SELECT productCode, SUM(quantityOrdered) AS
totalQuantityOrdered
        FROM      OrderDetails GROUP BY productCode) AS T
WHERE   T.totalQuantityOrdered >= 1000;
```

Description: Fetches the products whose collected orders are larger in quantity than 1000.

Implementation: Fetches the product names from the Products database query abbreviated as P, fetches the sum of the quantity ordered attribute based on the product code and sets it as the alias total quantity ordered specifies the order details query to be grouped by the product code and naturally joins as the alias T. Filters the results to only list results where the total quantity ordered attribute from the alias T is bigger than equal to 1000

Design & Implementation of PART II

The second part of the assignment is written as seven functions. The main function starts by setting up the structure of the input array buffer, allocates memory to both the structure as well as the character array inside and creates a temporary array for the standard input. Next some informational lines are printed to display an opening menu of sorts for the user. The while loop next is where all of the inputs for the shell gets "recorded". It will take the standard input from the terminal and check whether it's a query, or if it's a command. It does this by sending the input to both the "is command" function and the "command end function". The "is command" function checks if the first character is a "." as that would specify it being a command, it would then have its newline character at the end get cut out. If it's not a command, then that means it's a query and the loop will run until a semi colon is presented in the input.

After the main function has specified the inputs classification it calls to the parse command function which goes through all of the set-up commands for the shell or passes it on to the parse input function. There are four commands accepted by the shell which is; help, exit, quit and open. Help prints out a display of the four commands describing its use and exit and quit turns off the shell. Open however requires another terminal input as the command accepts different databases as long as the path is given.

The parse input function opens the database given with the database buffer structure, as it has been specified earlier with the command "open". Next the SQLite3 function execute takes the database buffer together with a couple arguments and performs several SQLite steps at once. One of them being a call to the callback function which is the function returning the resulting code in a JSON format. Lastly an error test is made on the result.

Callback prints the columns of the database along with the intended data in the JSON format using a global first instance variable and a loop to display all the resulting information in order correctly.

Discussion

There exist several flaws in the code as it is up to the user to trigger certain events. For instance, if the database given is wrong or not given the code does not have any way of figuring this out and will simply return empty or with an SQL syntax error. If the user starts the sequence with a "." the code will interpret it as a command, fail to understand it and try to parse it as a query. The result will be empty since the query parsed will be read with the ".".

Conclusion

The assignment and its tasks have been completed whereas task 1 has had its content explained or tried as per the requirement. The shell for the database application in C is capable of running the SQL queries, with options described through the display and a readme file. Plus the printed results comply with the JSON format restriction given. There has also been included a makefile to run the program.